

Earned Autonomy at the *Industrial Edge*

Most plants do not have an AI problem. They have an authority problem: nobody has written down what an agent is allowed to do to a machine, and under what evidence.



ONE SHIFT OF ALERTS AT A 40-PERSON PLANT. THE ONE THAT MATTERED IS IN GREEN.

AUTHOR

Bilal A. Khan

Fractional CTO, Stragentech

WRITTEN FOR

Operations and engineering leaders at \$10M–\$150M manufacturers

LENGTH

18 minutes · 6 figures · 24 references

01 / THE EVIDENCE

The bottleneck moved downstream. It did not disappear.

Engineering organizations pushed AI coding adoption up 65% and got roughly 8% faster software delivery. Teams with heavy AI use closed 21% more tasks while code review time climbed 91%.

Those two findings come from DX and Faros, and they describe the same event from opposite ends of the pipeline.^{1,2} Generation got cheap. Verification did not. Writing code was only about 16% of a developer's day to begin with, so making that part faster moves the delivery number by single digits while loading the review queue.³

Then there is the controlled result. METR ran a randomized trial with 16 experienced open-source developers across 246 real tasks in repositories they had worked in for about five years. Going in, the developers forecast a 24% time saving. Coming out, they reported a 20% saving. The stopwatch said tasks took 19% longer.⁴

+65%

rise in AI adoption across engineering orgs

DX

+8%

improvement in software delivery speed

DX

+91%

increase in code review time at high adoption

FAROS

19%

slower on real tasks in a randomized trial

METR

Set aside the headline. The durable finding is the 39-point spread between what practitioners believed about their own speed and what was measured. Skilled people, on their own code, were wrong about the direction of the effect.

A pull request sits still until somebody merges it. A setpoint change does not.

That is the reason this matters more in operations than in software. An agent on a plant floor does not emit diffs. It emits actions: a work order, a dispatch, a speed change, a valve command, a hold on a lot. Same stochastic engine, same failure signature of output that looks right and is not, and a blast radius measured in scrap, downtime, and occasionally people.

Practitioners already sense this. In one survey, 96% of developers said they do not fully trust AI-generated code and 61% said it often looks correct but proves unreliable.⁵ Useful instinct, useless control. "Do not fully trust" cannot be implemented. Thresholds, envelopes, expiry timers, and audit trails can.

The staffing version of the same mistake

Gartner expects 60% of organizations to run smaller software engineering teams by 2029, and warns that cutting junior roles in response to AI hollows out the talent pipeline.⁶ Hold that thought until section O2, failure F6. The people who handle the rare, ugly cases are made by handling the ordinary ones.

Steve Bennett of Wireless Logic framed the endpoint of this argument in EE Times: the winners will be the organizations that can say exactly where they draw the line and hold it.⁷ This paper is about how to draw that line as an engineering artifact rather than a policy statement, and how to move it deliberately as evidence accumulates.

Six ways agentic AI fails on a real plant floor

None of these are model quality problems. Every one of them is an architecture decision that nobody made on purpose.

F1 Threshold transplant

A confidence cutoff gets tuned while the output is a dashboard, then carried over when the output becomes an actuator. The number 0.85 means something very different when it produces a chart than when it produces a command to a 400-ton press. Confidence and consequence are separate axes, and most deployments collapse them into one.

F2 Correct reasoning over a sensor that is lying

Brownfield telemetry is where this bites. Two accelerometers on the same gearbox report in different units after a retrofit, and the tag description was never updated. The agent's chain of reasoning is sound, its citation of the data is accurate, and its conclusion is wrong. Fluent output over bad input is harder to catch than an obvious error, because it survives review.

F3 Alarm laundering

A typical mid-market plant runs a few hundred alerts a shift with a false positive rate north of 70%. Point an agent at that stream and it will produce five confident narratives instead of three hundred noisy ones. The false positive rate did not fall. It got compressed into prose, where it is no longer countable. Reviewers who used to audit data now audit summaries.

F4 Horizon decay

METR's time-horizon work found that the task length agents handle at 80% reliability runs four to six times shorter than at 50%.⁸ Failure also compounds non-linearly with task length, and adding scaffolding does not repair that uniformly.⁹ A six-step remediation is not six one-step problems. Plan the loop around the 80% number, or around whatever reliability your process actually needs.

F5 Instructions hidden in the maintenance record

Work orders, vendor PDFs, supplier emails, and technician notes are all untrusted text that your agent reads. OWASP's 2026 list for agentic applications catalogs goal hijack, tool misuse, and memory poisoning as the leading categories, with the governing principle stated as least agency: grant the minimum autonomy the task requires.¹⁰ In IT the worst case is data exfiltration. In OT the tool at the end of the chain writes to a PLC.

F6 Skill atrophy, on schedule

Lisanne Bainbridge described this in 1983 and it has not aged a day: automating the routine work leaves operators only the rare, difficult cases, while removing the daily practice that built the judgment

those cases require.¹¹ An agentic ops program that hits its efficiency targets and quietly retires the practice loop has bought a year of savings and sold five years of capability.

Every one of these is addressable. None of them are addressed by a better model, a longer prompt, or a vendor demo. They are addressed by where you put the boundaries and what evidence moves them.

Four planes, and only one is allowed to be clever

The workable pattern separates the stochastic part of the system from everything that touches a machine, and puts a deterministic checkpoint between them.

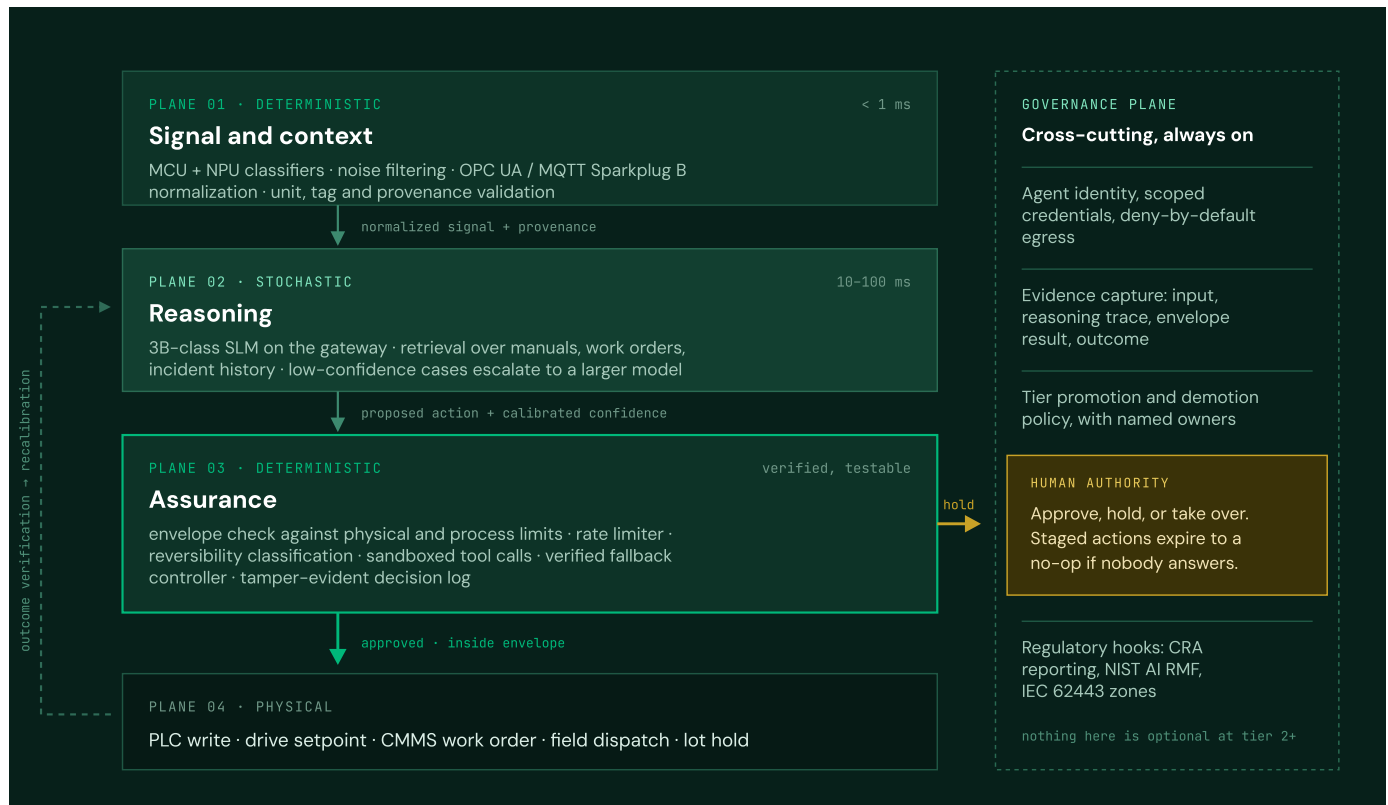


FIGURE 1 The stochastic plane proposes. The deterministic planes decide what reaches metal. The escalation path is a first-class output, not an error condition.

Plane 01: signal and context

Filtering, normalization, and provenance. This is unglamorous and it is where the return hides. Most mid-market plants are running first-generation IoT on top of equipment older than the network it sits on, which means duplicate tags, drifted calibrations, and units that changed during a retrofit nobody documented.

One rule holds the plane together: nothing reaches the reasoning layer without a source, a unit, and a timestamp you would be willing to produce in a warranty dispute. Sparkplug B and OPC UA give you the vocabulary. Enforcement is yours to build.

Plane 02: reasoning

This is the only stochastic component in the design. NVIDIA's research group argued the case for small models here better than most: agentic systems mostly invoke a language model to do a narrow, repetitive job, and for those jobs small models are sufficiently capable, operationally better suited, and far cheaper.¹² A 3B-class model on a gateway, fine-tuned on your incident history and your manuals, will beat a frontier model reached over a flaky cellular link on every axis that matters in a plant.

Structure it hierarchically. Run the small local model first and escalate only what it cannot handle confidently. Measured across five devices and three datasets, hierarchical inference held a target accuracy while cutting latency up to 73% and device energy up to 77% against on-device-only inference; adding early exit cut both further.¹³ The commercial argument arrives with the engineering one: escalation costs money and bandwidth, so the routing policy is also the cost model.

Escalation is a design decision, not a fallback

The routing rule that sends a case to a bigger model is the same rule that sends a case to a human. Build one policy, express it once, and apply it in both directions. Two separate escalation mechanisms drift apart within a quarter.

Plane 03: assurance

Control engineering solved a version of this problem before the current AI wave started. The Simplex architecture, formalized by Lui Sha, pairs a high-performance controller of uncertain reliability with a verified baseline controller and a monitor that hands authority to the baseline the moment safety conditions come under threat.¹⁴ That pattern has since been extended to machine-learned controllers, where design-time formal verification is not available and runtime assurance takes its place.¹⁵

Apply it directly. The language model is the high-performance controller. Your PID loops, interlocks, and standard operating procedures are the verified baseline. The assurance plane is the monitor, and it does five jobs: check the proposed action against the physical and process envelope, rate-limit it, classify its reversibility, execute it in a sandbox with scoped credentials, and write an immutable record of what was proposed, what was allowed, and what happened next.

Plane 04 and the governance rail

Actuation is whatever writes to the physical world, including the ones people forget to count: a CMMS work order that dispatches a technician at 2am, or a hold that idles a line. Governance runs alongside all four planes rather than sitting at the top of a diagram, because identity, evidence, and authority have to be present at every hop or they are present at none.

Autonomy is earned per action, not granted per system

"Is this agent autonomous?" is the wrong question. The answerable version: for this specific action, on this specific asset, at this measured confidence, what is the agent allowed to do without a human?

That question has two independent inputs. How reliable is the judgment, and how bad is it if the judgment is wrong. Plot them against each other and the policy writes itself.



FIGURE 2 Autonomy routing. Confidence alone never justifies an action; severity alone never forbids one. The pair decides, and the cell assignment is a reviewable artifact rather than a hallway agreement.

What each tier actually means

Tier	The agent	The human	Evidence to promote out
T0 observe	Annotates and correlates. Emits no recommendation.	Works exactly as before, with better context.	200+ annotated cases graded for usefulness by the crew.
T1 recommend	Proposes a ranked action with its evidence and its confidence.	Decides and executes. Records agreement or override with a reason.	Agreement rate and override reasons stable across 60 days and two crews.
T2 stage and confirm	Prepares the action fully, then waits. Expires to a no-op on timeout.	Approves inside a time box. Silence means no action.	Zero envelope violations; approval latency inside the process window.
T3 act and report	Executes inside a pre-approved envelope, rate-limited, then reports.	Reviews after the fact, on a schedule, with sampling.	Measured error at the operating point below the agreed risk budget.
T4 act within envelope	Runs the loop. Reports exceptions and drift only.	Owens the envelope, the risk budget, and the quarterly recertification.	Nothing. T4 is the ceiling, and it is reserved for reversible actions.

The two rules that keep this honest

Demotion is automatic; promotion is manual. Drift detection, an envelope violation, or a sensor calibration event drops the affected action to T1 without a meeting. Moving back up takes a human signature and fresh evidence.

Silence is not consent. A staged action that nobody approves must expire into a no-op. Systems that default to execute after a timeout have quietly promoted themselves to T3 while the org chart still says T2.

Vendors sell tier 4 across the board because tier 4 demos well. In brownfield environments it is rarely the right answer, and the honest version of the pitch is narrower: tier 4 for reversible, high-frequency, well-

instrumented actions, tier 2 for anything that costs real money to undo, and a documented path between them.

Trust is a measurement, and most deployments never take it

A confidence score from a language model is a number the model produced. Calling it a measurement is a category error that has quietly authorized a lot of bad decisions.

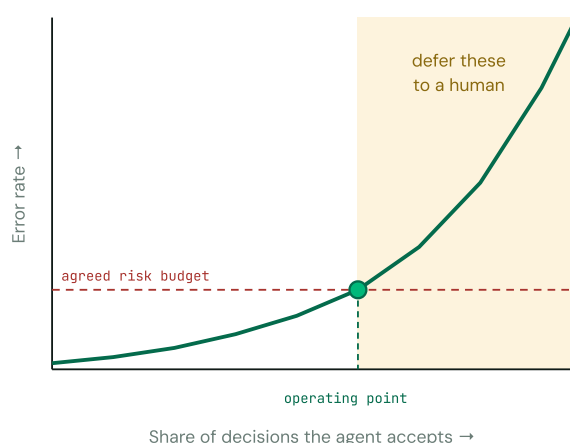
Recent work on selective prediction makes the point precisely: token entropy on its own is not enough to decide when a model should answer and when it should stand down.¹⁶ The number correlates with correctness under favorable conditions and stops correlating exactly when conditions get unfavorable, which is when you needed it.

The fix is old and well understood outside the AI industry. Give the system an explicit third option beyond right and wrong: abstain. Chow formalized the reject option in the 1970s, and modern selective prediction turned it into a risk-coverage problem where the objective is low error on the cases you accept, with everything else deferred.¹⁷

Conformal prediction supplies the missing guarantee. Rather than trusting the model's self-reported confidence, you calibrate on held-out data and get prediction sets with finite-sample coverage that holds without assuming anything about the model's internals.¹⁸ In clinical triage work, pairing conformal sets with cost-aware deferral cut error on retained cases by roughly 47% to 50% at fixed coverage, including on out-of-distribution splits.¹⁹ The same shape of problem appears on a plant floor: high volume, rare positives, asymmetric costs, and a human expert who is expensive but available.

PANEL A • WHERE TO SET THE LINE

Error rises as the agent accepts more of the decisions.



PANEL B • WHETHER TO BELIEVE THE SCORE

A model saying 90% should be right 90 times in 100.



FIGURE 3 Two questions that decide whether an autonomy tier is defensible. Panel A sets the coverage at which measured error stays inside the risk budget. Panel B checks whether the confidence score means anything at all. Both curves move after a retrofit, a product-mix change, or a season.

Rare failures break the naive version of this

Equipment failures are rare events, and conformal coverage computed across the whole population will happily meet its target while systematically under-covering the class you actually care about. Class-conditional (Mondrian) conformal prediction fixes that. In a benchmark across 15 imbalanced real-world datasets, class-conditional calibration restored minority-class coverage by an average of 61.7 percentage points over the marginal version.²⁰ Calibrate per failure mode, not per fleet.

Coverage guarantees assume a world that plants do not live in

Conformal guarantees rest on exchangeability between calibration and deployment data. A plant violates that assumption every time it changes product mix, replaces a bearing supplier, or crosses into summer. Treat empirical coverage as a live production metric, recalibrate on a rolling window, and set drift detection to trip an automatic tier demotion.

The seven numbers worth reporting

Accuracy is not on this list. Accuracy without a coverage figure describes nothing, because a system that answers 4% of cases can post any accuracy it likes.

Metric	Why it earns its place on the wall
Coverage at target risk	The share of decisions the agent takes while staying inside the risk budget. This is the actual productivity number.
Selective risk	Error measured only on the cases the agent accepted. The number your operations lead cares about.
Envelope violation rate	Proposals the assurance plane blocked. Should be non-zero in testing and zero in production.
Override rate and reasons	A rising override rate is drift. A falling one with no accuracy change is complacency.
Calibration error over time	Panel B as a monthly trend line rather than a one-time acceptance test.
Mean time to human takeover	How long the plant runs on a bad decision before a person intervenes. Tests the alerting path, not the model.
Cost-weighted error	A false dispatch and a missed bearing failure are not the same event. Weight them and the threshold moves on its own.

The model is a component. The harness is the product.

Two different things get called a harness in this field, and an industrial deployment needs both: the runtime scaffolding that surrounds the model in production, and the test rig that decides whether it is allowed to run at all.

The runtime harness

Everything around the model that governs what it sees, what it can call, and what happens when it is wrong. The empirical case for taking it seriously keeps strengthening. One recent framework separated task execution from state management and independent progress verification, giving each execution round a fresh context, and lifted pass rate on a long-horizon benchmark from 51.8% to 80.7% using the same underlying models.²¹ Researchers have started arguing that comparing agents without disclosing their harness is close to meaningless, because that is where reliability lives.²²

Translated to a plant, the harness carries four responsibilities.

- **State lives outside the context window.** The authoritative record of what the line is doing belongs in your historian and your MES, not in a conversation transcript that degrades as it grows.
- **Tool schemas encode physical limits.** A setpoint tool that cannot accept a value outside the process band is worth more than a paragraph of instructions telling the model to stay in the band.
- **Verification checks the world, not the claim.** After an action, read the sensor. The agent reporting success is an assertion; the vibration coming down is evidence.
- **Recovery paths are written before deployment.** What happens on a partial failure, a timeout mid-sequence, or a network partition with a command in flight. Decide it in design review, because the alternative is deciding it at 3am.

RUNTIME ASSURANCE LOOP

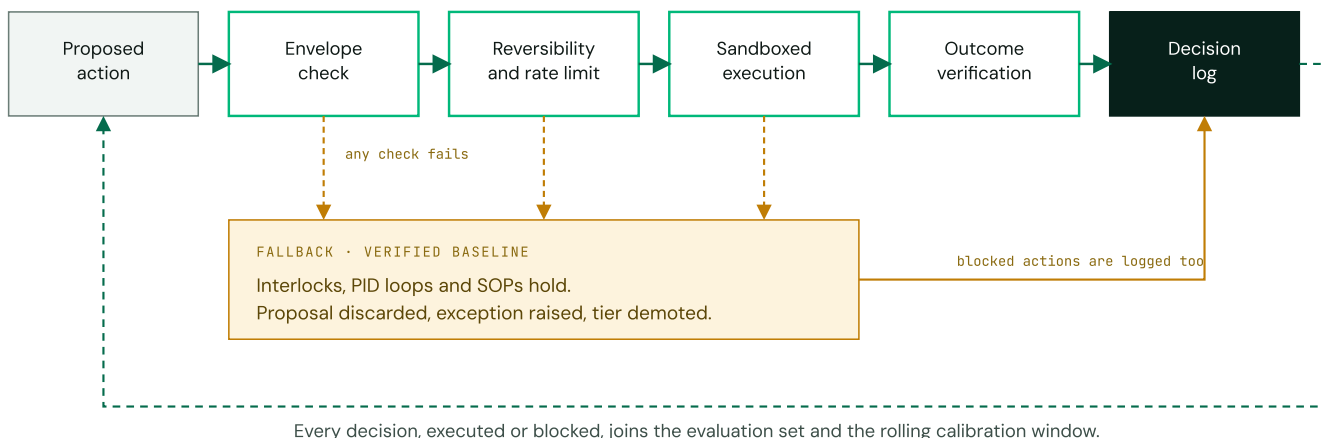


FIGURE 4 The Simplex pattern applied to agent tool calls: an unverified high-performance component proposes, a verified baseline stands ready, and a deterministic monitor decides which one holds authority. The blocked-action path is instrumented as carefully as the success path.

The evaluation harness

Generic benchmarks tell you nothing about your gearboxes. What you need is a plant-specific regression suite that runs on every change and produces the evidence that moves a cell in Figure 2.

01 Incident replay.

Last 18 to 24 months of real incidents, replayed with the outcome withheld. Grade the agent's proposal against what actually resolved it. This is also the cheapest way to find out that half your incident records have no resolution field filled in.

02 Fault injection.

Stuck-at sensor values, swapped units, clock skew, duplicate tags, a network partition mid-command. The correct behavior is abstention with a clear reason. Improvisation is a failure even when the improvisation happens to work.

03 Adversarial content.

Work orders, technician notes, and vendor PDFs seeded with instruction-shaped text. OWASP's agentic list ranks goal hijack and tool misuse at the top for good reason.¹⁰ The agent should ignore the instruction and flag the document.

04 Envelope probes.

Proposals engineered to land just outside the physical or process limit. The assurance plane catches every one, or the tier does not move.

05 Shadow scoring.

The agent runs in parallel with the crew for a defined window, deciding nothing. That record is your promotion evidence and your calibration set at the same time.

A model swap is a change to a safety-relevant component

Version bumps, prompt edits, tool schema changes, and gateway firmware updates all invalidate your calibration. Run the suite. Manufacturing already knows how to do this; it is the same discipline that governs a change to a control program, applied to an artifact that happens to be probabilistic.

Checks and balances that survive an audit

Governance in agentic ops comes down to one structural question: which identity proposes, which approves, which executes, and which audits. Collapse any two of those into the same identity and the controls become decoration.

Function	Held by	Failure when it collapses
Propose	The agent, with its evidence and confidence attached.	Proposals without evidence cannot be reviewed, only believed.
Approve	A named human at T1 and T2; a pre-signed envelope at T3 and T4.	An agent that approves its own actions has one control, not two.
Execute	A scoped service identity with parameterized, typed tool access.	Shared credentials make attribution impossible after an incident.
Audit	A separate reader with no write path, on a schedule with sampling.	Self-reported logs are testimony from an interested party.

Three controls do most of the work in an OT environment. Agent identities stay distinct from human identities, so the log answers who without ambiguity. Network egress is denied by default, because an agent that cannot reach an unexpected endpoint cannot be talked into using one. Tools are parameterized rather than general, so the agent calls `set_speed(line=3, rpm=<bounded>)` instead of holding a shell on the historian.

Where the existing standards already cover you

Very little of this needs inventing. Manufacturing has spent forty years building the frameworks; the work is mapping agent behavior onto them.

Framework	What it governs in an agentic deployment
NIST AI RMF + AI 600-1	The lifecycle wrapper: govern, map, measure, manage. The generative AI profile adds the risk categories specific to language models. ²⁴
ISA/IEC 62443	Zones and conduits. The reasoning plane belongs in its own zone with an inspected conduit to the control zone, and the gateway is a new asset in your inventory.
IEC 61508 / ISO 13849	Safety functions. An agent stays outside the safety function entirely. Anything with a safety integrity level keeps its existing certified path and the agent gets no authority over it.
OWASP Top 10 for Agentic Applications	Agent-specific threat modeling: goal hijack, tool misuse, identity abuse, memory poisoning, rogue agents. Use it for red-team scoping before go-live. ¹⁰
EU Cyber Resilience Act	Product security obligations for anything connected that you place on the EU market, including the vulnerability reporting clock now running. ²³

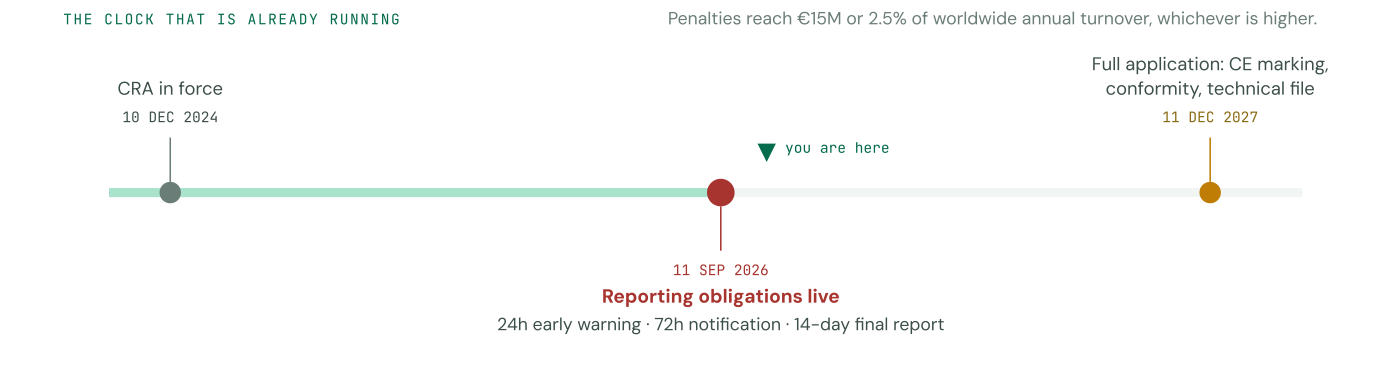


FIGURE 5 If you place connected products on the EU market, the reporting obligation started this month and the conformity deadline is fifteen months out. Products already on the market are pulled in when they undergo a substantial modification, which is a question worth asking your counsel about field-updated agent behavior.

The kill switch that actually works

Every agentic deployment has one on the slide. Ask three questions of yours. Who is authorized to pull it, by name and at 2am on a Sunday. How long does the plant take to reach a known-good state after it is pulled, measured with a stopwatch rather than estimated. When was it last tested end to end, in production, with the crew that would actually use it. A switch tested once at commissioning is a documented intention.

Ninety days, one workflow, three gates

Mid-market plants have an advantage over enterprise here that rarely gets named: one meeting can hold everyone whose approval matters. Use it. The sequence below assumes a single named workflow and refuses to widen until the gates pass.

DAYS 1-30 Frame	DAYS 31-60 Instrument	DAYS 61-90 Operate
One workflow, named, with a dollar figure attached to it Blast radius map for every action class the agent could take Measured baseline: false positive rate, cost of a miss, cost of a false dispatch Signal inventory with units, tags and provenance	Shadow mode running. The agent decides nothing Calibration set built from shadow decisions, per failure mode Evaluation harness: incident replay, fault injection, adversarial content, envelope probes Envelopes written as code. Tier grid drafted and signed	Tier 2 live on one cell or one line, nowhere else Weekly promotion review with the named owner in the room Freed hours pre-committed to a named outcome, in writing Demotion triggers armed and tested at least once
GATE 1 Blast radius map signed, baseline set	GATE 2 All probes caught, calibration curve plotted	GATE 3 30 days at T2, zero violations, before T3

FIGURE 6 The gates matter more than the phases. Skipping gate 1 is how a deployment ends up unable to prove it changed anything, because nobody measured the plant before the agent arrived.

Pre-commit the freed capacity before you free it

The step most often skipped, and the one that decides whether savings reach the P&L. Decide in writing, before deployment, what the recovered hours are for: a maintenance backlog, a second shift avoided, a certification that has been waiting eighteen months. Attach a name. Capacity that is not pre-committed gets quietly reabsorbed into the same queue it came from, and six months later the CFO asks a fair question that nobody can answer.

Keep the practice loop alive

Bainbridge's warning has an operational answer. Rotate a share of agent-handled cases back to humans on purpose, even when the agent would have been right. Run quarterly manual-recovery drills on the workflows the agent now owns. Budget for it, schedule it, and treat the cost as insurance against the day the agent hits a case nobody has seen since 2019.

Eight questions for anyone selling you an agent

These are not gotchas. A good vendor answers all eight in twenty minutes and will be relieved someone finally asked.

01 Show me your tier grid.

If autonomy is a single on-off setting in the product, it has not yet met a regulated plant.

02 What is the abstention rate on data like mine, and where do those cases go?

A system that never abstains is either extraordinary or uncalibrated. The odds favor the second.

03 How is confidence calibrated, and when was it last recalibrated?

Listen for held-out data and a rolling window. Listen for silence on the word calibration.

04 What stops an action that violates a physical limit, and does that code run in your product or mine?

The envelope belongs on my side of the boundary. Vendors who agree are the ones who have been through an incident.

05 Which harness produced your benchmark numbers, and can I run the same suite against my incident history?

Benchmark scores without a harness description are marketing.²²

06 What is the rollback for each action class, and what has actually been rolled back in a live plant?

Ask for the incident, not the design. The answer tells you whether anything has ever gone wrong in front of them.

07 What identity does the agent use, what can it reach, and what is denied by default?

If the answer involves a shared service account with broad access, you have found your first finding.

08 If our contract ends, what happens to the plant?

Model weights, gateway hardware, envelope definitions, calibration sets, and decision logs. Establish now which of those you own.

Where you draw it, and what moves it

Every architectural choice in this paper reduces to one artifact: a written statement of what an agent may do to a machine, under what evidence, with whose signature, and what automatically takes that permission away.

Most plants running agentic pilots today do not have that artifact. They have a vendor default, a Slack thread, and an assumption held slightly differently by the maintenance lead and the IT director. That works until the first case where the agent was confident and wrong, which is also the first case where somebody asks who approved it.

So a question worth taking into your next operations review. Pick the single action your agents would take most often. Can you say, in writing, what confidence and what consequence class would let that action run without a person in the loop, and who signs when that boundary moves? If the sentence does not exist yet, that is the artifact worth building first, ahead of any model selection.

ABOUT

Stragentech

Stragentech is a fractional CTO practice building Agentic Operational Intelligence for industrial manufacturers, managed service providers, and the private equity firms that own them. The work draws on 25 years of production systems: AWS IoT Analytics and Project Kuiper enterprise engineering at Amazon, legacy modernization and agentic workflow automation at Boeing / Jeppesen ForeFlight, and the CTO seat at a PE-backed industrial computing company.

Bilal A. Khan is based in Seattle and works remotely nationwide. Engagements start with an Operational Intelligence Assessment and a prioritized roadmap.

stragentech.com · info@stragentech.com · linkedin.com/in/bilalakhan

REFERENCES

1. DX. *AI productivity gains more modest than expected*. getdx.com
2. Faros AI. *AI in software engineering: task throughput and review time*. faros.ai
3. DX. *New data: AI's impact on engineering velocity is more modest than expected*. getdx.com
4. Becker, J., Rush, N., Barnes, E., Rein, D. (2025). *Measuring the Impact of Early-2025 AI on Experienced Open-Source Developer Productivity*. METR. [arXiv:2507.09089](https://arxiv.org/abs/2507.09089)
5. Sonar. *The State of Code: developer survey report*. sonarsource.com
6. Gartner (2026). *Gartner predicts 60% of organizations will adopt smaller software engineering teams by 2029*. gartner.com
7. Bennett, S. (2026). *How Much Impact Will AI Have on IoT Software Engineering?* EE Times. eetimes.com

8. Kwa, T. et al. (2025). *Measuring AI Ability to Complete Long Software Tasks*. METR. arXiv:2503.14499
9. Survey of tool-use, planning and reasoning failures in LLM agents (2026): failure compounds non-linearly with task length and added scaffolding does not uniformly improve reliability. arXiv:2607.05775
10. OWASP GenAI Security Project (2025). *Top 10 for Agentic Applications 2026 (ASIO1–ASIO10)*. genai.owasp.org
11. Bainbridge, L. (1983). *Ironies of Automation*. *Automatica* 19(6), 775–779.
12. Belcak, P. et al. (2025). *Small Language Models are the Future of Agentic AI*. NVIDIA Research. arXiv:2506.02153
13. Moothedath, V. et al. (2024). *Exploring the Boundaries of On-Device Inference: When Tiny Falls Short, Go Hierarchical*. arXiv:2407.11061
14. Sha, L. (2001). *Using Simplicity to Control Complexity*. *IEEE Software* 18(4), 20–28.
15. Runtime Safety Assurance for Learning-enabled Control of Autonomous Driving Vehicles (2021). arXiv:2109.13446
16. Entropy Alone is Insufficient for Safe Selective Prediction in LLMs (2026). arXiv:2603.21172
17. Geifman, Y., El-Yaniv, R. (2017). *Selective Classification for Deep Neural Networks*. NeurIPS 30. Building on Chow's reject-option formulation.
18. Vovk, V., Gammerman, A., Shafer, G. (2005). *Algorithmic Learning in a Random World*. Springer. Split conformal methods per Papadopoulos et al. (2002).
19. Conformal selective prediction with cost-aware deferral for safe clinical triage under distribution shift (2026). *Scientific Reports*. nature.com
20. Cost-Sensitive Conformal Prediction and Human-in-the-Loop Abstention for Imbalanced High-Stakes Decision Support (2026). arXiv:2607.27143
21. LongHorizon-Harness: Advancing Long-Horizon Agents for Real-World Tasks (2026). arXiv:2608.01964
22. Stop Comparing LLM Agents Without Disclosing the Harness (2026). arXiv:2605.23950
23. European Commission. *Cyber Resilience Act (Regulation EU 2024/2847) and reporting obligations*. digital-strategy.ec.europa.eu
24. NIST. *AI Risk Management Framework (AI 100-1)* and *Generative AI Profile (NIST AI 600-1, 2024)*. nist.gov